# Pipelining

(Computer Architecture: Chapter 3 & Appendix C)

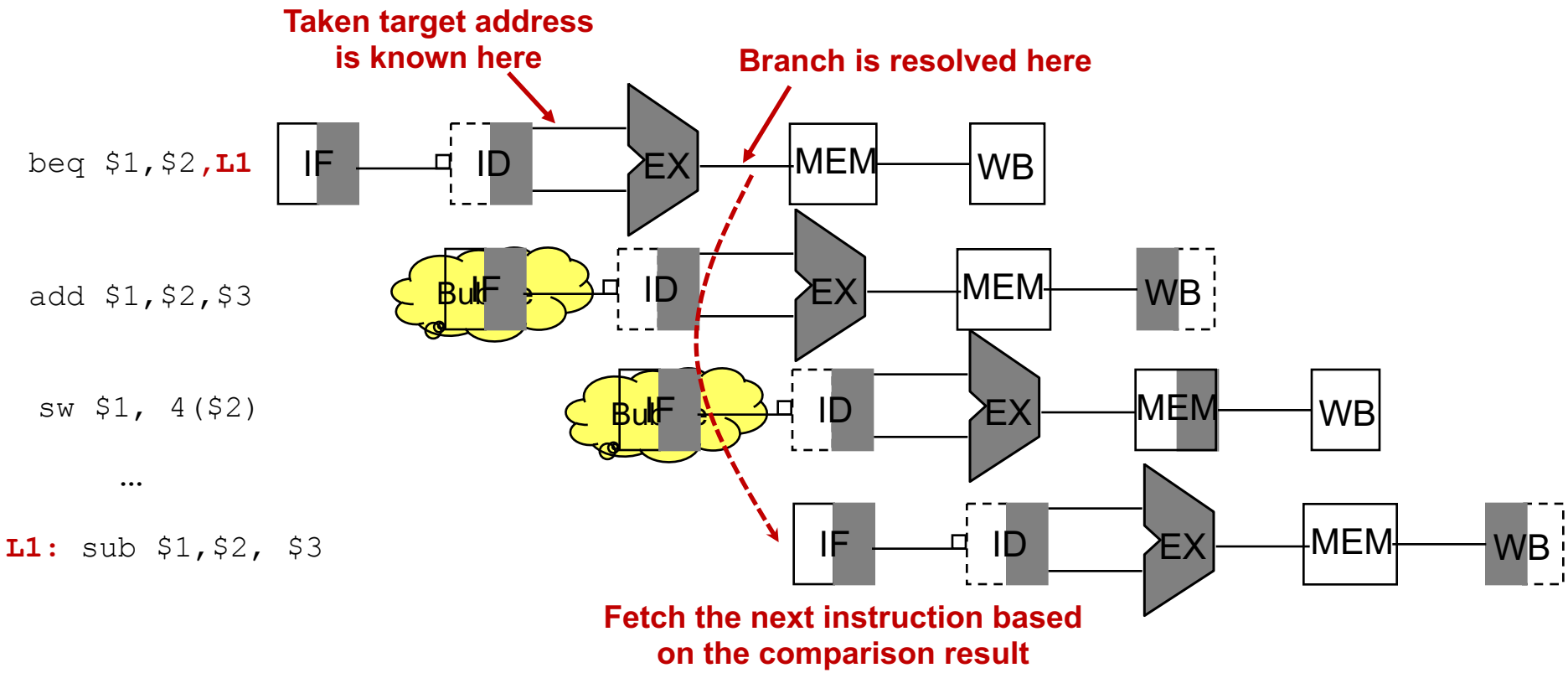
**Yanyan Shen**

Department of Computer Science and Engineering

Shanghai Jiao Tong University

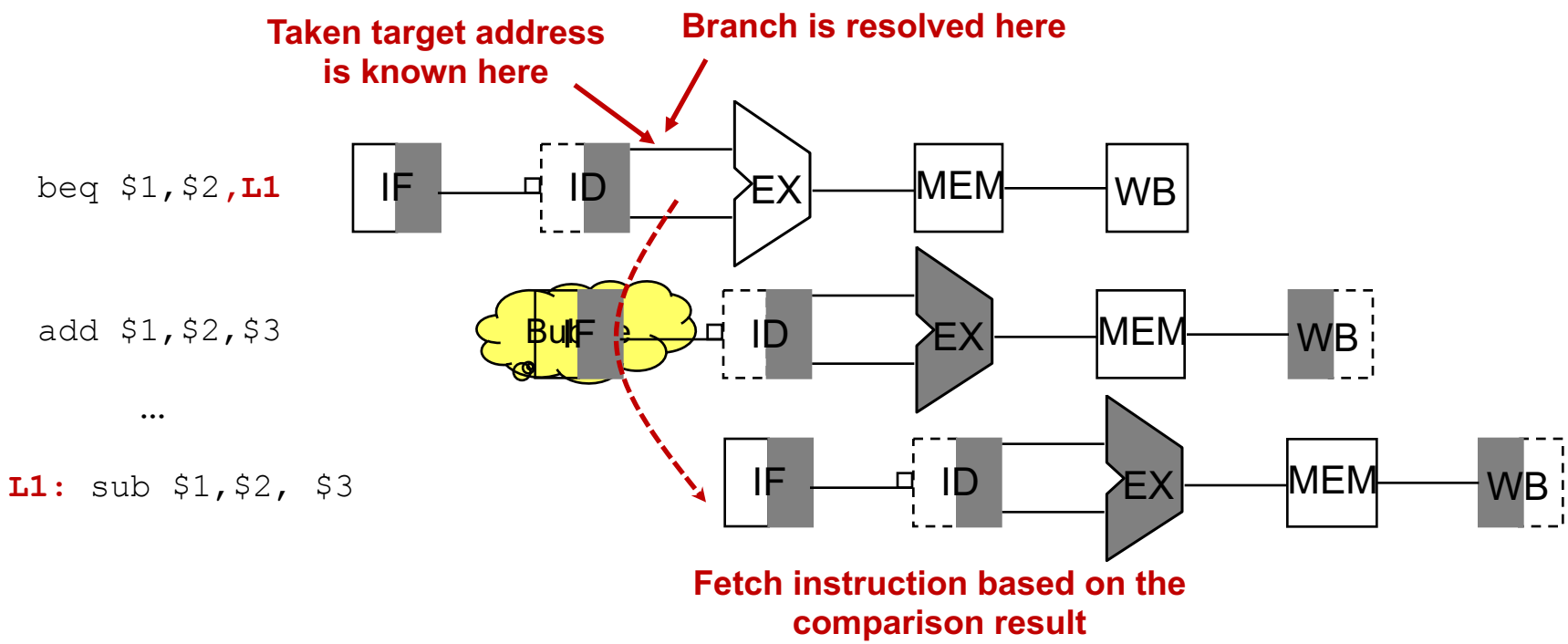
# Control Hazard

- Branch determines the flow of instructions
- Fetching the next instruction depends on the branch outcome
  - Pipeline can't always fetch correct instruction
  - Branch instruction is still working on ID stage when fetching the next instruction



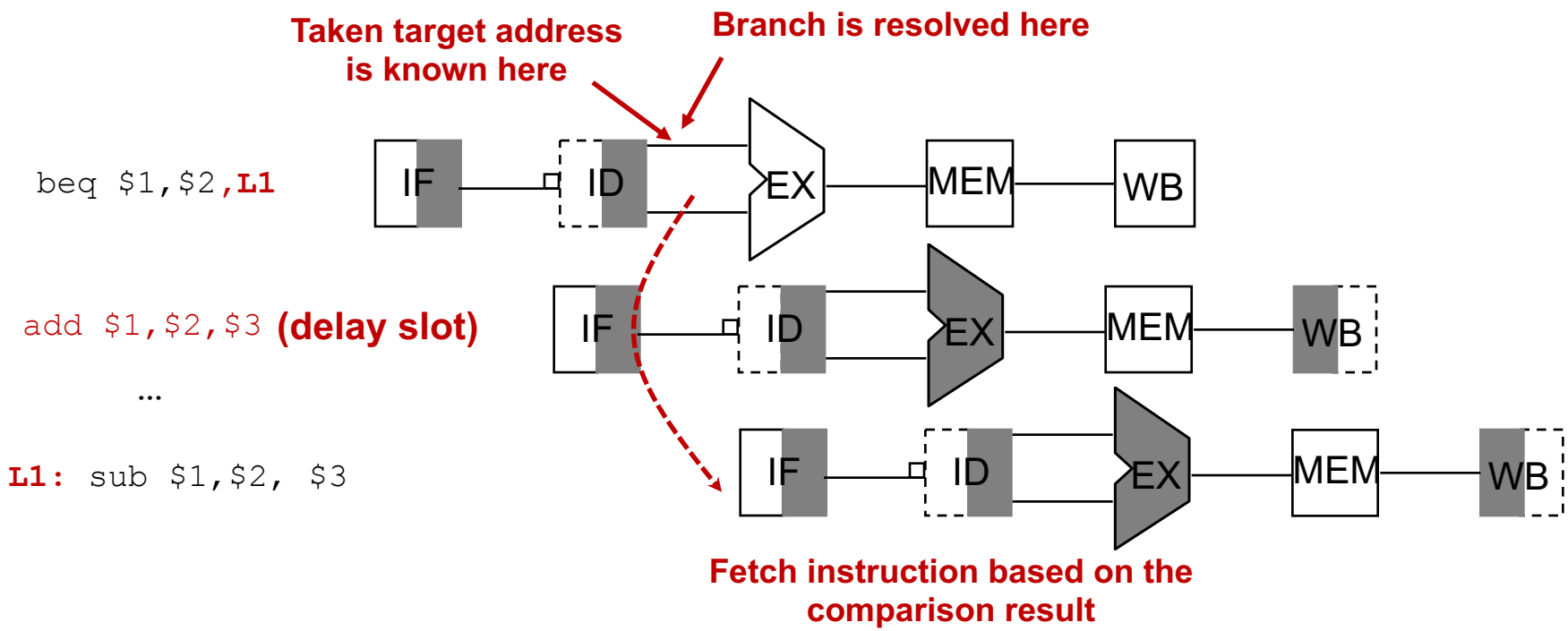
# Reducing Control Hazard

- To reduce 2 bubbles to 1 bubble, add hardware in ID stage to compare registers (and generate branch condition)



# Delayed Branch

- Many CPUs adopt a technique called the ***delayed branch*** to further reduce the stall
  - **Delayed branch** always executes the next sequential instruction
    - The branch takes place after that one instruction delay
  - **Delay slot** is the slot right after a delayed branch instruction



# Delay Slot (Cont.)

- Compiler needs to schedule a useful instruction in the delay slot, or fills it up with `nop` (no operation)

```
// $s1 = a, $s2 = b, $3 = c
// $t0 = d,  $t1 = f
a = b + c;
if (d == 0) {f = f + 1;}
f = f + 2;
```

→

```
     add $s1,$s2, $s3
     bne $t0,$zero, L1
     nop  //delay slot
     addi $t1, $t1, 1
L1:  addi $t1, $t1, 2
```

**Can we do better?**

```
     bne $t0, $zero, L1
     add $s1,$s2,$s3 // delay slot
     addi $t1, $t1, 1
L1:  addi $t1, $t1, 2
```

Fill the delay slot with a useful and valid instruction

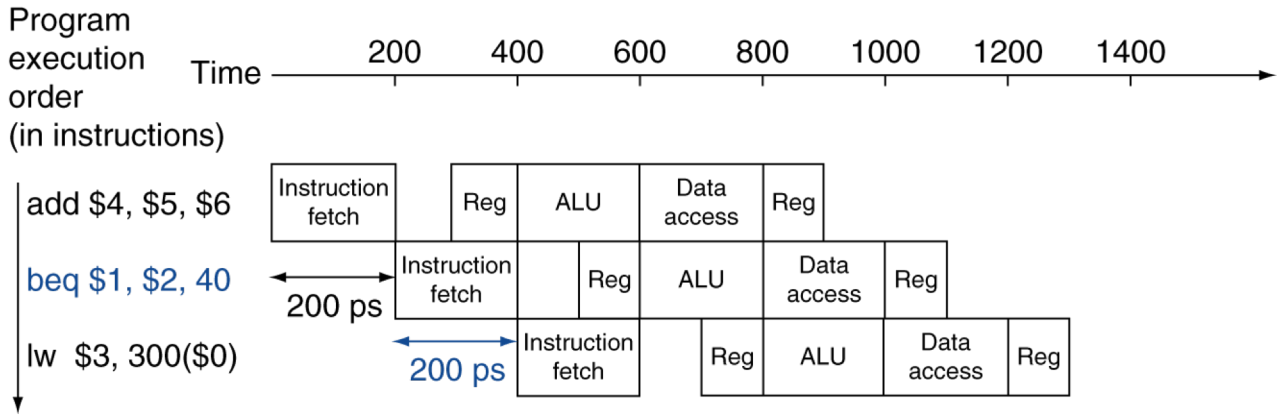
# Branch Prediction

- Longer pipelines (implemented in Core 2 Duo, for example) can't readily determine branch outcome early
  - **Stall penalty** becomes unacceptable since branch instructions are used so frequently in the program

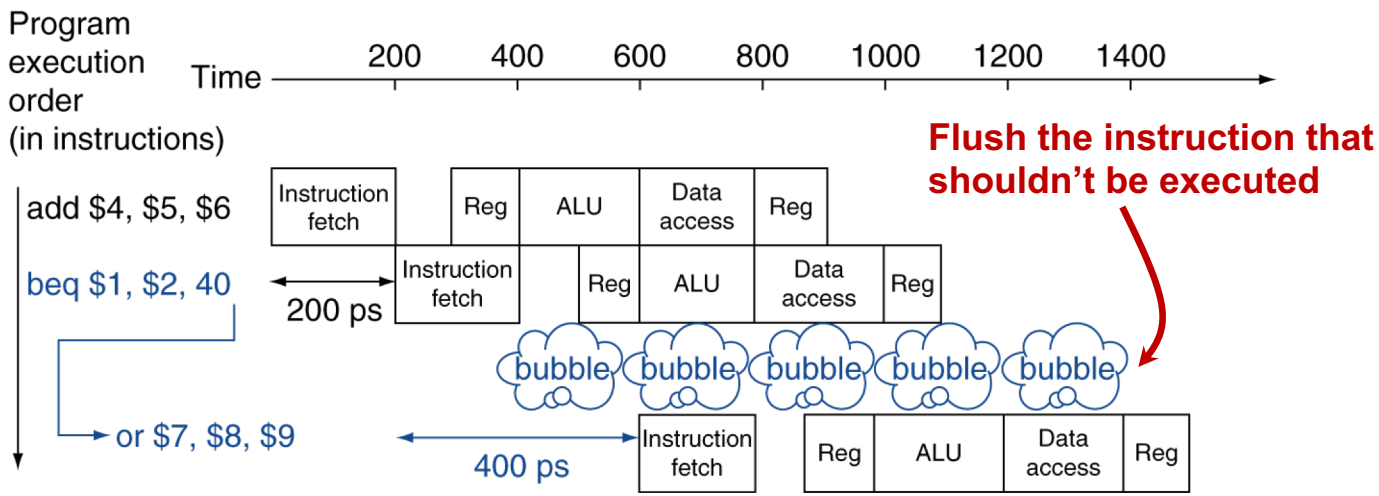
- **Solution: Branch Prediction**
  - Predict the branch outcome in hardware
  - **Flush** the instructions (that shouldn't have been executed) in the pipeline if the prediction turns out to be wrong
  - Modern processors use **sophisticated branch predictors**

# MIPS with Predict-Not-Taken

**Prediction correct**

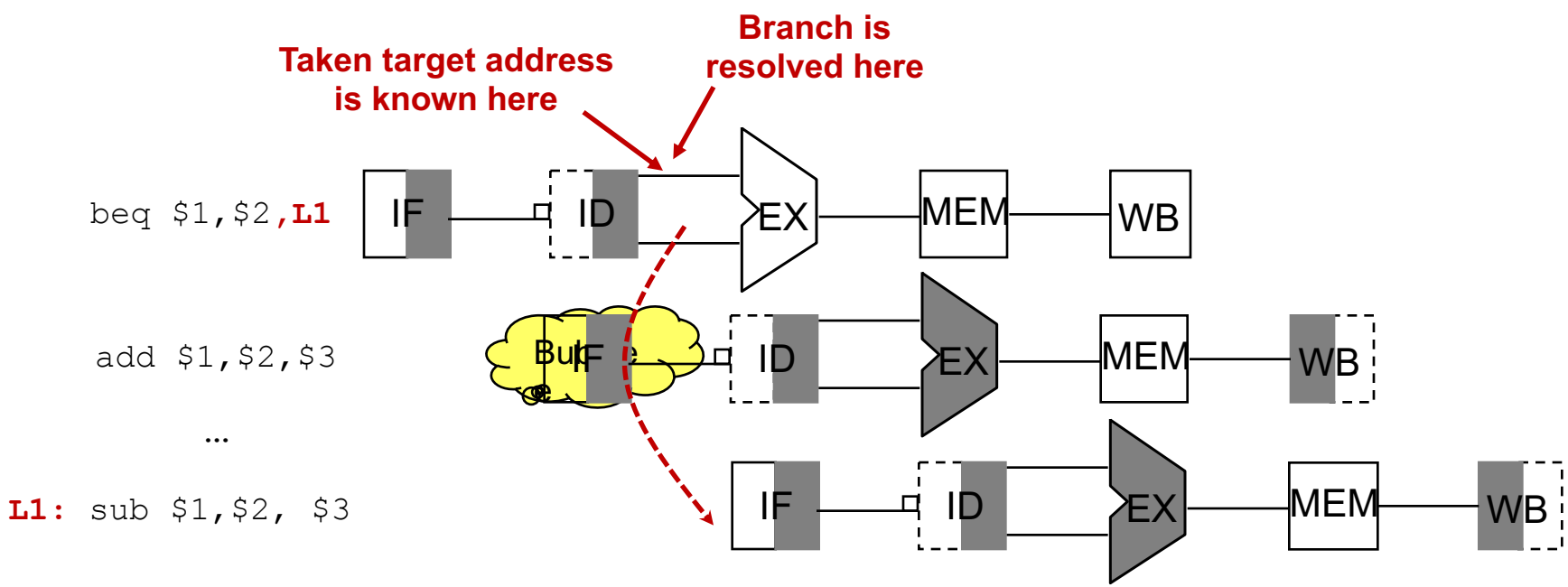



**Prediction incorrect**

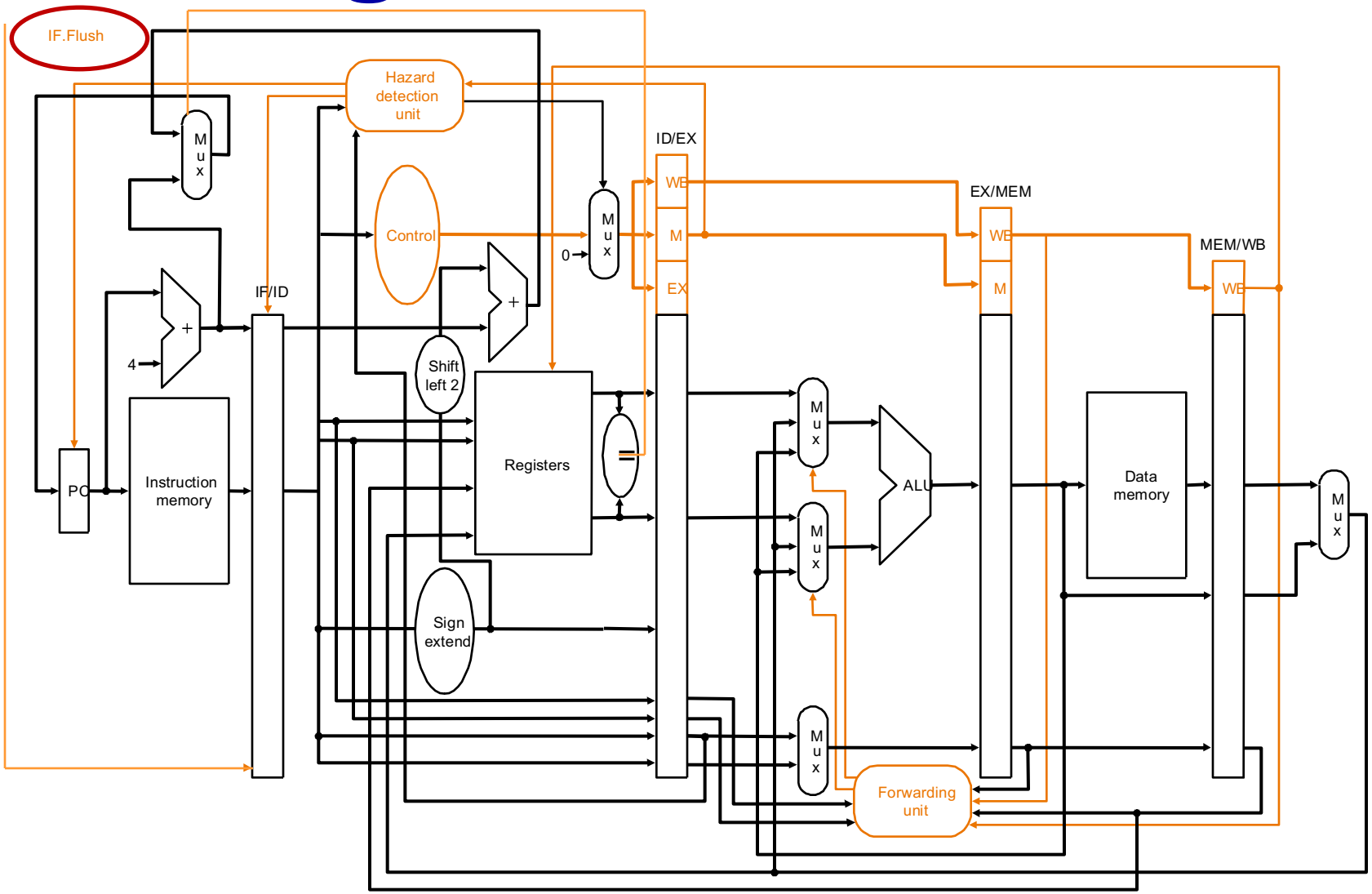


# Alleviate Branch Hazards

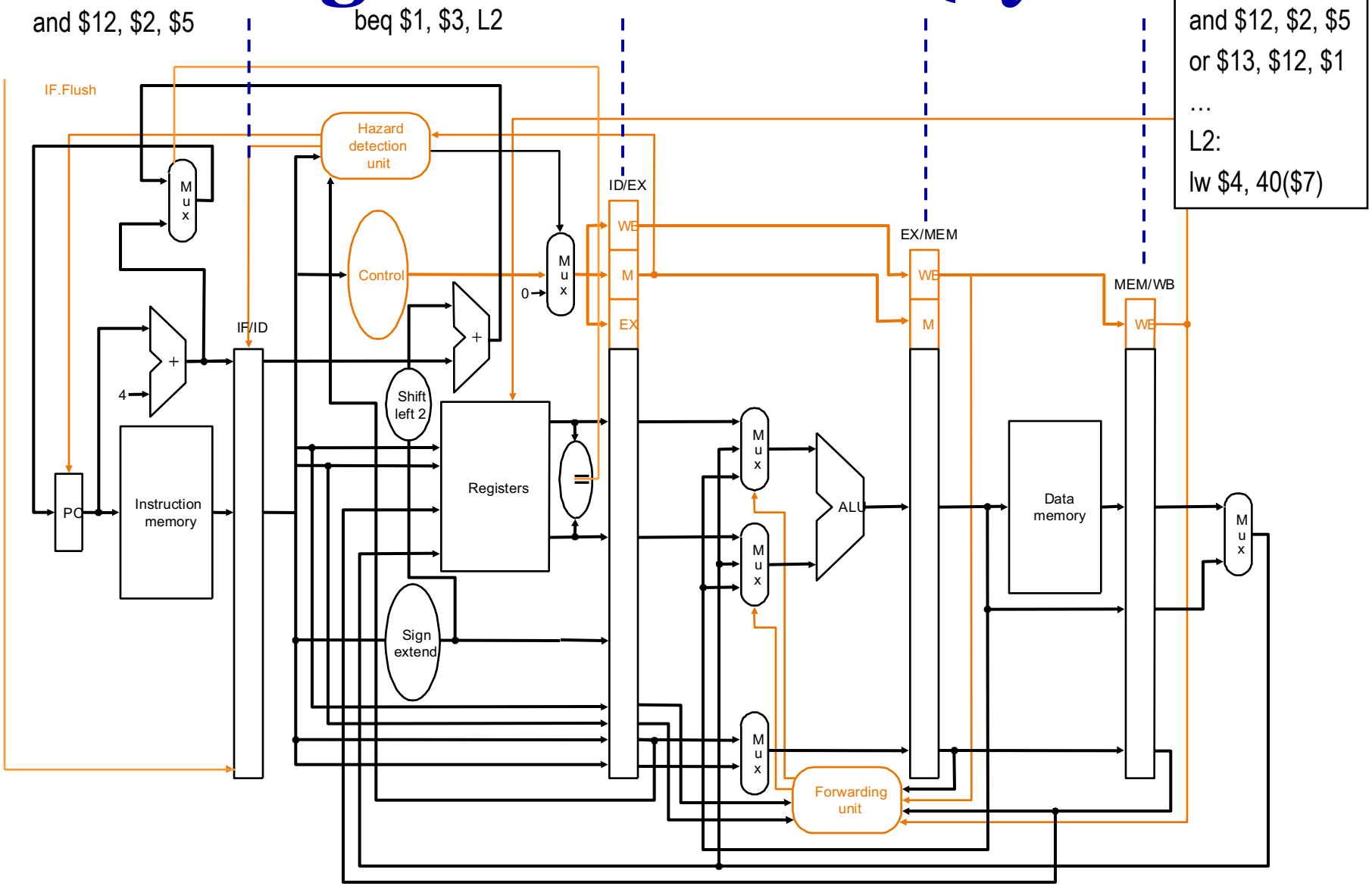
- Reduce penalty to 1 cycle
  - Move the branch compare to the ID stage of pipeline
  - Add an adder to calculate the branch target in ID stage
  - Add the IF.flush signal that zeros the instruction (or squash) in IF/ID pipeline register



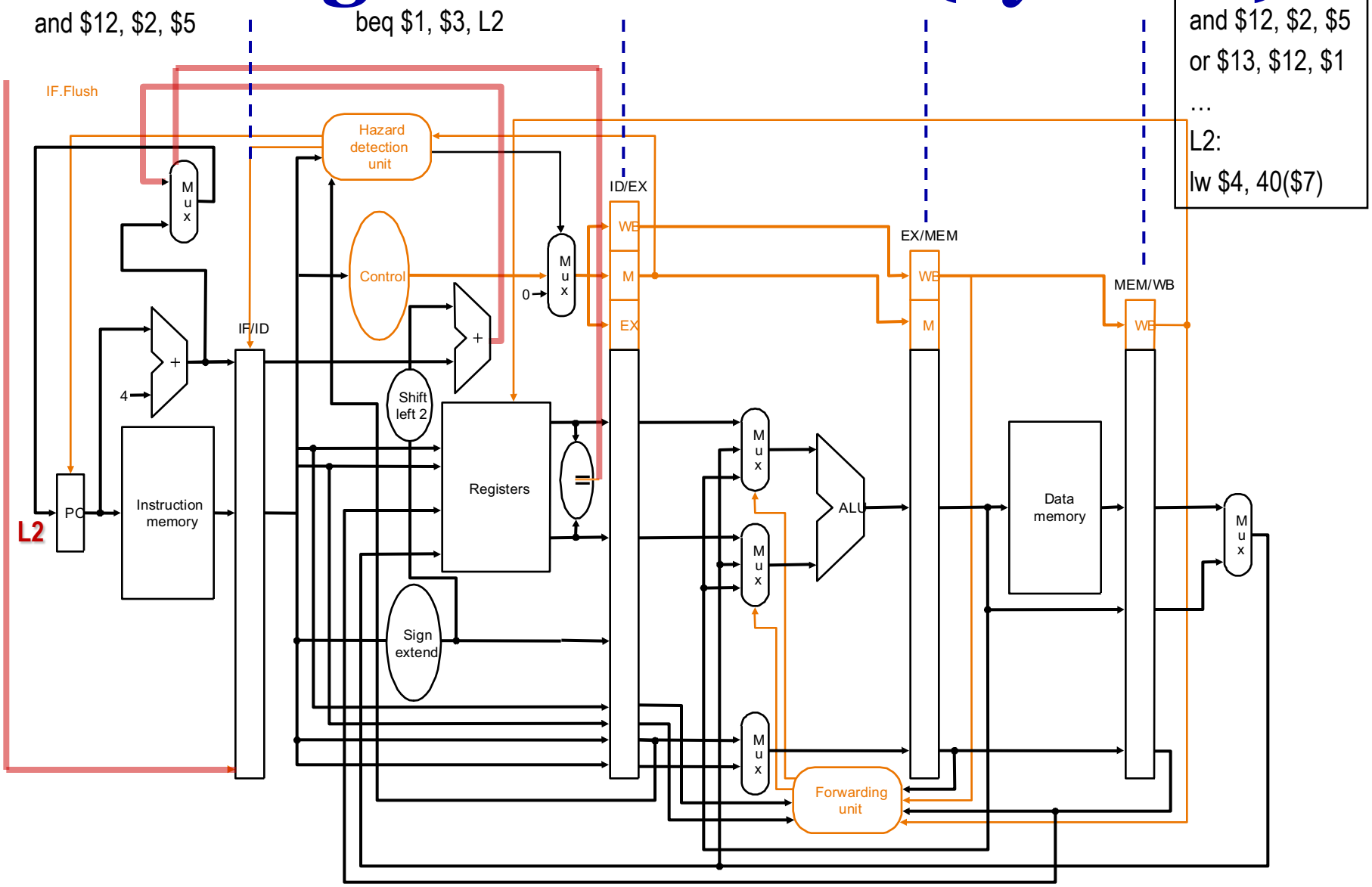
# Flushing Instructions



# Flushing Instructions (cycle N)

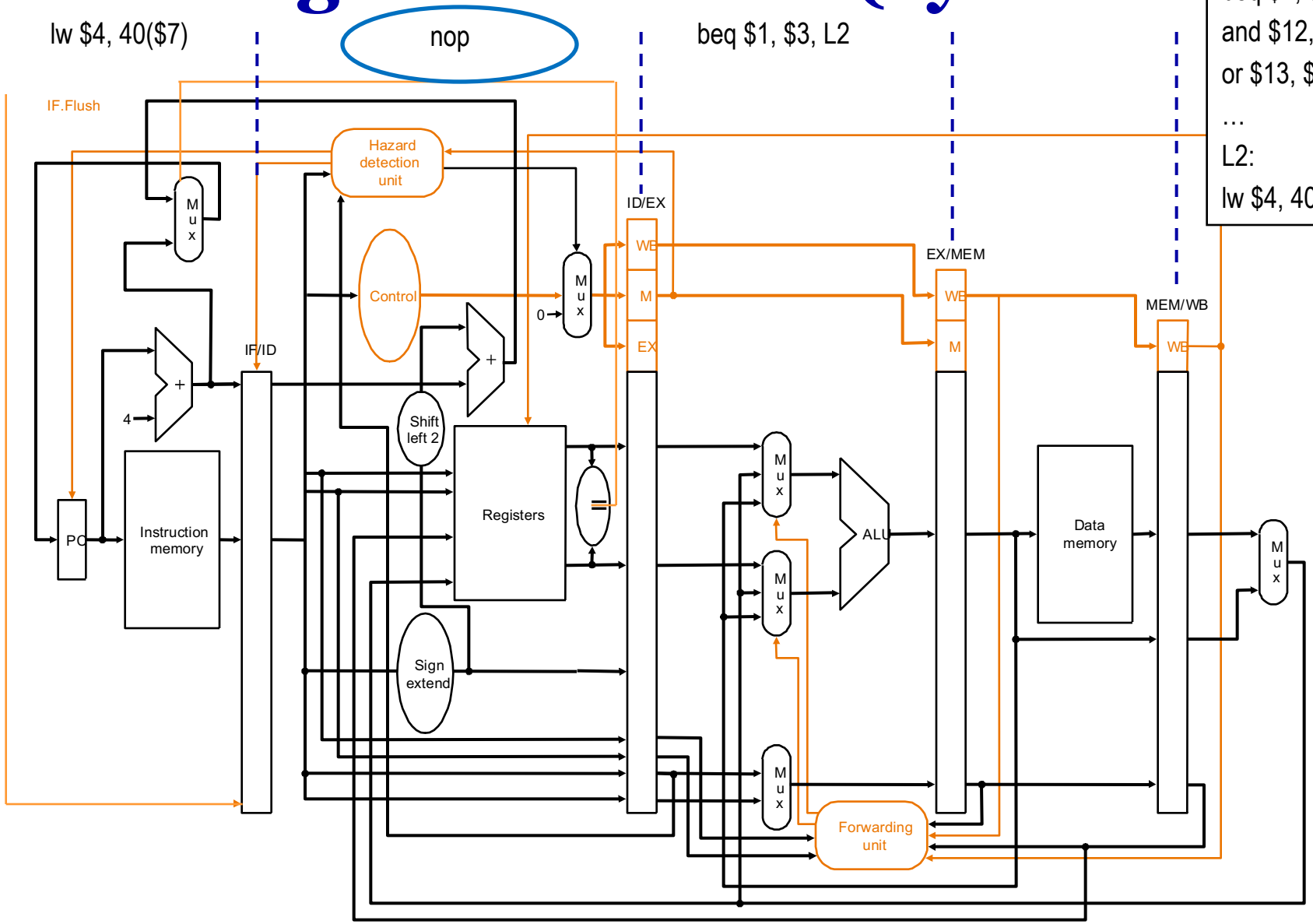


# Flushing Instructions (cycle N)



# Flushing Instructions (cycle N+1)

beq $1, $3, L2
and $12, $2, $5
or $13, $12, $1
…
L2:
lw $4, 40($7)



# Outline

- Introduction to Pipelining
- How Pipeline is Implemented
- Pipeline Hazards
- C.4 Exceptions
- Handling Multicycle Operations

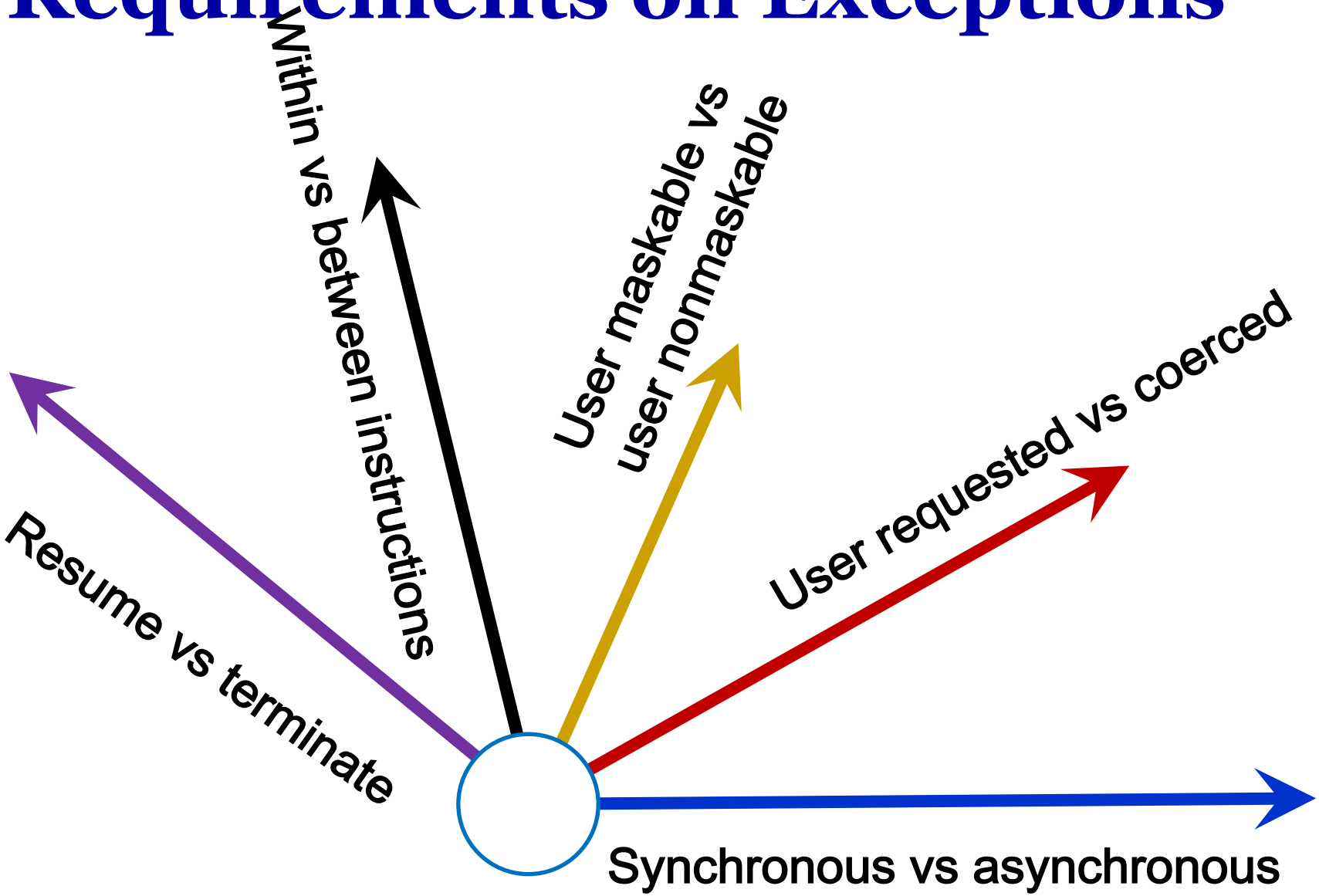
# Exceptions

- **Exceptions** describe those situations where the normal execution order of instruction is changed!
  - may force the CPU to **abort** the *instructions in the pipeline* before they complete!
- Some other used terminologies for "exception"
  - Interrupt
  - Fault
- We use exception!

# Types of Exceptions

- I/O device request
- Invoking an OS service for a user program
- Tracing instruction execution
- Breakpoint (programmer requested interrupt)
- Integer arithmetic overflow
- FP arithmetic anomaly
- Page fault (not in main memory)
- Misaligned memory accesses
- Memory protection violation
- Using an undefined instruction
- Hardware malfunctions
- Power failure

# Requirements on Exceptions



# Classifications

| Exception type | Synchronous vs. asynchronous | User request vs. coerced | User maskable vs. nonmaskable | Within vs. between instructions | Resume vs. terminate |
|---|---|---|---|---|---|
| I/O device request | Asynchronous | Coerced | Nonmaskable | Between | Resume |
| Invoke operating system | Synchronous | User request | Nonmaskable | Between | Resume |
| Tracing instruction execution | Synchronous | User request | User maskable | Between | Resume |
| Breakpoint | Synchronous | User request | User maskable | Between | Resume |
| Integer arithmetic overflow | Synchronous | Coerced | User maskable | Within | Resume |
| Floating-point arithmetic overflow or underflow | Synchronous | Coerced | User maskable | Within | Resume |
| Page fault | Synchronous | Coerced | Nonmaskable | Within | Resume |
| Misaligned memory accesses | Synchronous | Coerced | User maskable | Within | Resume |
| Memory protection violations | Synchronous | Coerced | Nonmaskable | Within | Resume |
| Using undefined instructions | Synchronous | Coerced | Nonmaskable | Within | Terminate |
| Hardware malfunctions | Asynchronous | Coerced | Nonmaskable | Within | Terminate |
| Power failure | Asynchronous | Coerced | Nonmaskable | Within | Terminate |

# Exceptions

- **Exceptions** describe those situations where the normal execution order of instruction is changed!
  - may force the CPU to **abort** the *instructions in the pipeline* before they complete!
- Some other used terminologies for "exception"
  - Interrupt
  - Fault
- We use exception!

# Stopping and Restarting Execution

- The most ***difficult*** exceptions have two properties
- **(1)** they occur **within instructions** (i.e., in the middle of the instruction execution corresponding to EX or MEM pipe stages
- **(2)** they must be **restartable**

# Steps to Save Pipeline State

- **(1)** Force a **trap instruction** into the pipeline on the next IF
- **(2)** Until the trap is taken, turn off all writes *for the faulting instruction* and *for all instructions that follow in the pipeline*
  - This can be done by placing zeros into the pipeline latches of all instructions, starting with the instruction that generates the exception, but not those that precede that instruction
- **(3)** After the exception-handling routine in the OS receives control, it immediately *saves the PC of the faulting instruction*
  - This value will be used to return from the exception later

# Precise vs Imprecise Exceptions

If the pipeline can be stopped so that *the instructions just before the faulting instruction* are completed and *those after it* can be restarted from scratch, the pipeline is said to have precise exceptions

- Supporting precise exceptions is a requirement in many systems
- Any processor with demand paging or IEEE arithmetic trap handlers must make its exceptions precise

# Exceptions in MIPS Pipeline

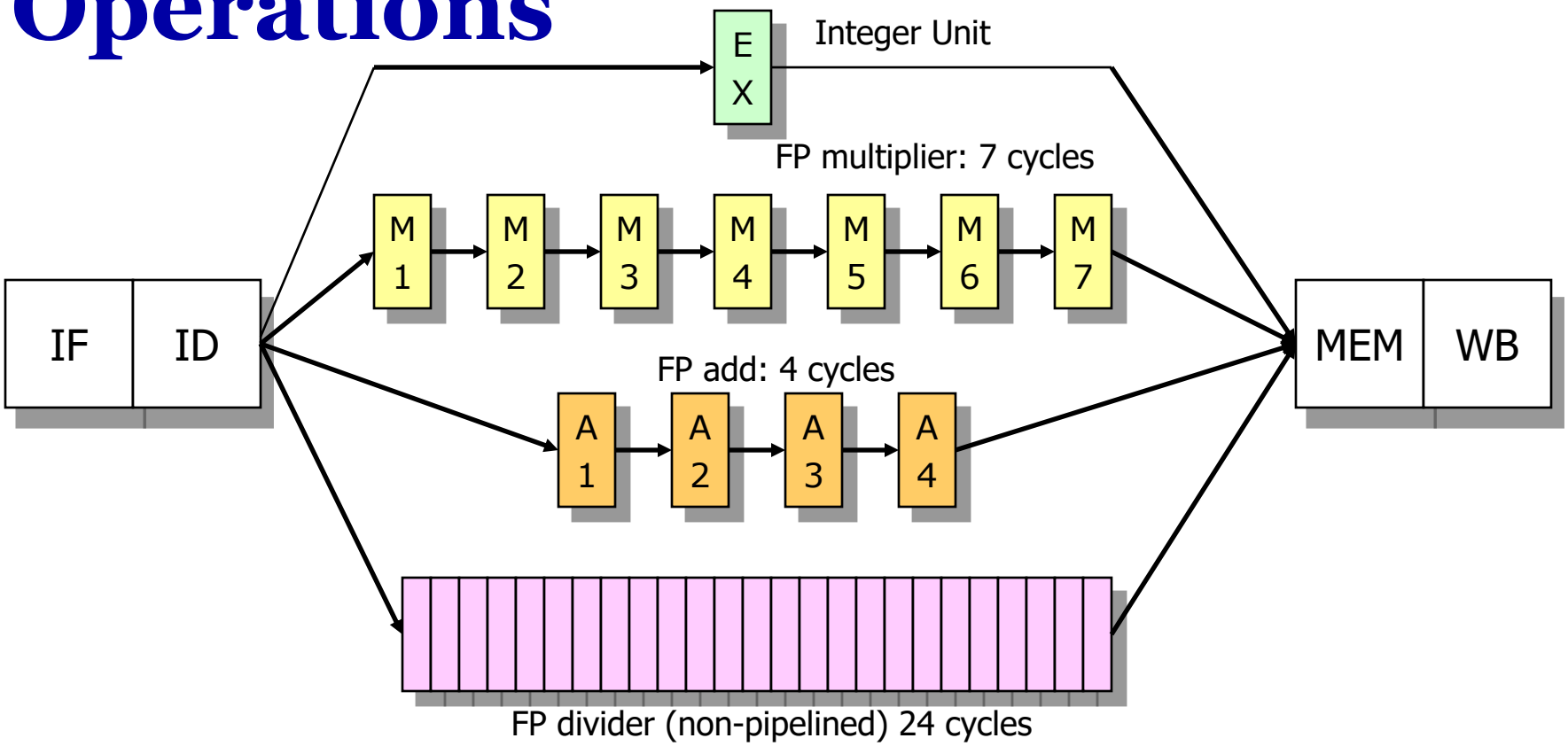
| Pipeline stage | Problem exceptions occurring |
|---|---|
| IF | Page fault on instruction fetch; misaligned memory access; memory protection violation |
| ID | Undefined or illegal opcode |
| EX | Arithmetic exception |
| MEM | Page fault on data fetch; misaligned memory access; memory protection violation |
| WB | None |

Exceptions may occur in different stages of a pipeline

# Outline

- Introduction to Pipelining
- How Pipeline is Implemented
- Pipeline Hazards
- Exceptions
- C.5 Handling Multicycle Operations

# Supporting Multiple FP Operations




- **Complicate bypass or forwarding**
- **Potential structural hazard**
- **Multiple (FP) instructions can complete at the same time**
  - **RF might need to be multi-ported**
  - **Ordering issue, who gets to update the register?**
- **Out-of-order completion/retirement: Precise exception issue**

Modified from Prof Sean Lee's Slide

# Bypassing & Forwarding

| Clock Cycles | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| L.D F4,0(R2) | IF | ID | EX | M | WB | | | | | | | | | | | | | |
| MUL.D F0,F4,F6 | IF | ID | S | M1 | M2 | M3 | M4 | M5 | M6 | M7 | M | WB | | | | | | |
| ADD.D F2,F0,F8 | IF | S | ID | S | S | S | S | S | S | A1 | A2 | A3 | A4 | M | WB | | | |
| S.D F2,0(R2) | IF | S | S | S | S | S | S | ID | EX | S | S | S | M | WB | | | | |

# Structural Hazards

| Clock Cycles | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| MUL.D F0,F4,F6 | IF | ID | M1 | M2 | M3 | M4 | M5 | M6 | M7 | M | WB |
| . . . . | | IF | ID | EX | M | WB | | | | | |
| . . . . | | | IF | ID | EX | M | WB | | | | |
| ADD.D F2,F4,F6 | | | | IF | ID | A1 | A2 | A3 | A4 | M | WB |
| . . . . | | | | | IF | ID | EX | M | WB | | |
| . . . . | | | | | | IF | ID | EX | M | WB | |
| L.D F2,0(R2) | | | | | | | IF | ID | EX | M | WB |

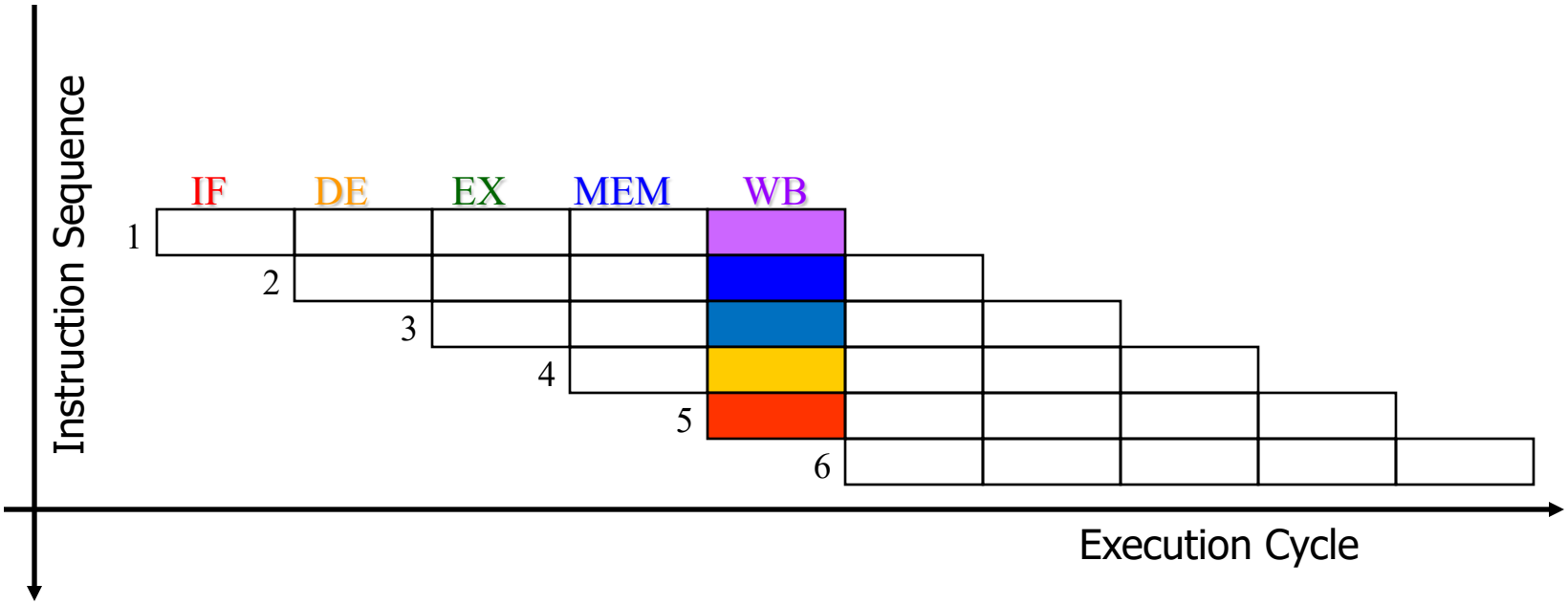
- Write to register file at the same cycle (cc11)
- Write to the same register (WAW)
- MEM in cc10

# Precise Exception Issue

```
DIV.D   F0,F2,F4      (exception!)
ADD.D   F3,F10,F8     (completed)
SUB.D   F12,F12,F14   (completed)
```

- **Precise exception**: If the pipeline can (or must) be stopped
  - All the instructions before the faulty (or intended) instruction must be completed
  - All the instructions after it must not be completed
  - Restart the execution from the faulty (or intended) instruction
- State must be consistent with the original program order
- Not straightforward with out-of-order completion

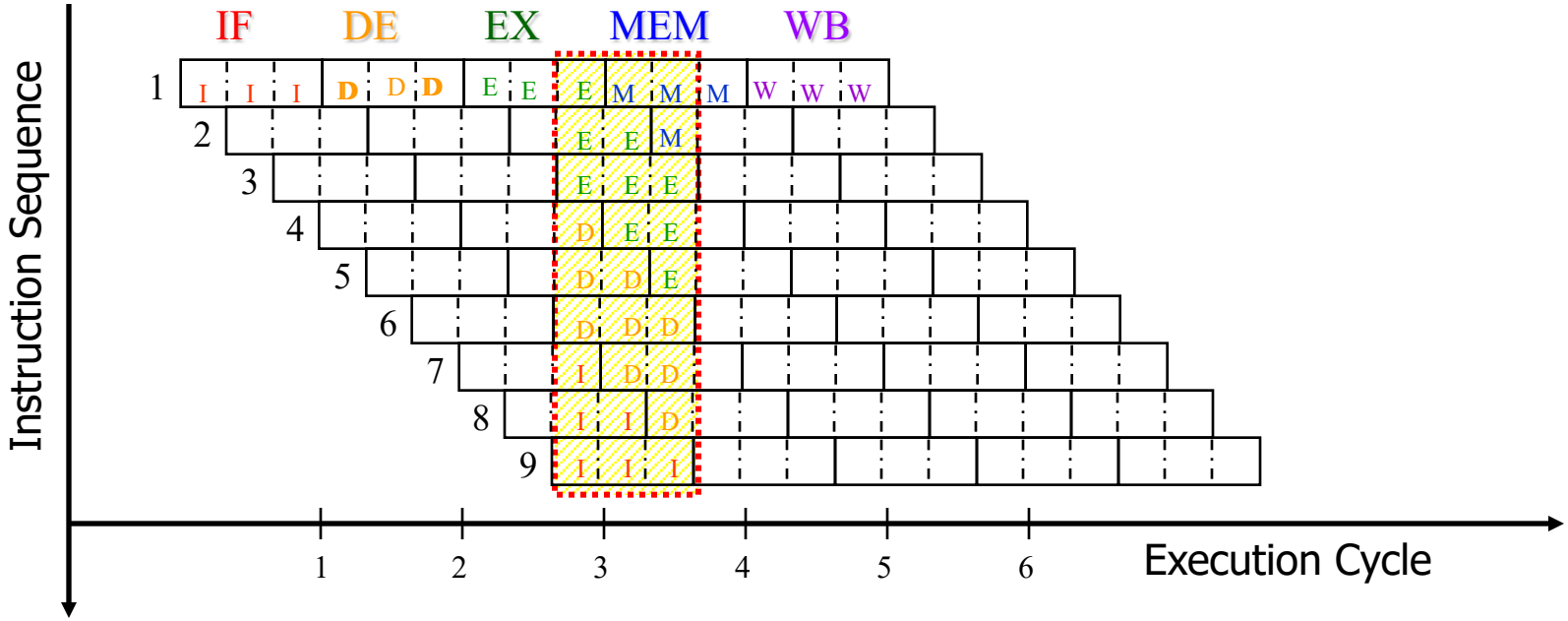
# Scalar Pipeline (Baseline)




Modified from Prof Sean Lee's Slide

# Superpipeline

- Deeper pipelining is called superpipelining
- Deeper pipeline allows for achieving higher clock rates




Modified from Prof Sean Lee's Slide

# End of Pipelining